Supervised Bayesian Matrix Factorization

The LB and YFB Parameterizations Compared:
Derivations, Supervision Strength and Prediction

Andrew Walther

November 2026

Contents

1 Summary **1**

2 The Two Models Side by Side **2**

3 Updates Side by Side **2**

4 Where the Balance Between Genomics and Survival Comes From **3**

5 What Happens When Survival Is Given More Weight **4**

6 Prediction for a New Patient **5**

7 LB Implementation Review **6**

8 Strengths, Weaknesses and Recommendation **7**

1 Summary

Both models factor expression as $Y = LF^\top + E$ and link survival through a Cox model. They differ in the linear predictor:

$$\text{LB: } \eta_i = \boldsymbol{l}_i^\top \boldsymbol{\beta} \qquad \text{vs.} \qquad \text{YFB: } \eta_i = \boldsymbol{y}_i^\top F \boldsymbol{\beta}.$$

1. **Which parameter survival supervises.** In LB the patient scores L appear in both likelihoods. In YFB the gene loadings F do. In each model the other factor matrix is updated exactly as in unsupervised EBMF.

2. **Whether survival supervises the factorization.** In both models it does in principle, through the dual-source update. In the YFB benchmark configuration used through October 2026 (`alpha_F = 0`) it did not: that configuration is unsupervised EBMF followed by a shrinkage Cox fit. The current YFB configuration (`canonicalize = TRUE`, weight w_S) restores the survival term in the F update.

3. **How strongly.** Survival's share of the dual-source precision falls with the number of genes in both models, roughly as $1/(pv_k)$ in LB and $1/(pv_k)^2$ in YFB, where v_k is the program's

variance share. In both it is largest for low-variance programs. At $p \approx 2{,}000$ and equal weights it is negligible for dominant programs. The share is a weight, not the size of the change: the relative change it makes in a loading falls roughly as $1/p$ in both.

4. **Prediction.** YFB scores a new patient exactly by projection, $\hat{\eta}^* = \boldsymbol{y}^{*\top} \bar{F} \bar{\boldsymbol{\beta}}$. LB needs the new patient's scores $\boldsymbol{l}^*$. Yusha's fold-in gives them as an EBNM posterior under the frozen training priors. It worked in a small simulation, but it cannot reproduce the survival contribution to the training scores.

5. **LB code.** The LB fitter runs and its test suite passes, but its default sign check inverted correctly oriented fits. This was fixed on 2026-11-06 (Section 7).

2 The Two Models Side by Side

	LB	YFB
Genomics model	$Y = LF^\top + E$	$Y = LF^\top + E$
Linear predictor	$\eta_i = \sum_k l_{ik}\beta_k$	$\eta_i = \sum_k z_{ik}\beta_k$, $z_{ik} = \sum_j y_{ij} f_{jk}$
Survival conditions on	latent scores L	observed Y through the loadings F
Joint likelihood	$p(Y \mid L, F, \tau)\, p(t, \delta \mid L, \boldsymbol{\beta})$	$p(Y \mid L, F, \tau)\, p(t, \delta \mid Y, F, \boldsymbol{\beta})$
Dual-source parameter	L $(n \times K)$	F $(p \times K)$
Genomics-only parameter	F	L
Coefficient interpretation	log HR per unit of score l_k	log HR per training SD of score z_k
New patient	needs $\boldsymbol{l}^*$: fold-in or projection	$\hat{\eta}^* = \boldsymbol{y}^{*\top} \bar{F} \bar{\boldsymbol{\beta}}$, exact
Scale fixed by	EBNM priors only	score standardization, $\mathrm{sd}(z_{\cdot k}) = 1$

The structural point. In LB, survival and expression are conditionally independent given L: both are noisy readouts of the same latent patient state. In YFB, survival is modeled given the measured expression, and the loadings play two roles: they define programs that reconstruct Y and they define the risk score. LB keeps the outcome out of the expression side of the model but has to infer $\boldsymbol{l}^*$ for a new patient. YFB gives exact risk scores but uses Y both as an outcome and as a covariate.

3 Updates Side by Side

Notation follows the two derivation documents: bars are posterior moments, R^{-k} the reconstruction residual without program k, W_{ii} and z_i the Cox working weight and response, z_i^{-k} the working response without the other programs, and w_S the survival weight ($w_S = 1$ in the LB derivation). Every update is EBNM$(x = B/A,\ s = 1/\sqrt{A})$.

	LB	YFB
$q(l_{ik})$	$A = \sum_j \tau_j \overline{f_{jk}^2} + w_S W_{ii} \overline{\beta_k^2}$ $B = \sum_j \tau_j R_{ij}^{-k} \bar{f}_{jk} + w_S W_{ii} z_i^{-k} \bar{\beta}_k$	$A = \sum_j \tau_j \overline{f_{jk}^2}$ $B = \sum_j \tau_j R_{ij}^{-k} \bar{f}_{jk}$
$q(f_{jk})$	$A = \tau_j \sum_i \overline{l_{ik}^2}$ $B = \tau_j \sum_i R_{ij}^{-k} \bar{l}_{ik}$	$A = \tau_j \sum_i \overline{l_{ik}^2} + w_S \overline{\beta_k^2} \sum_i W_{ii} y_{ij}^2$ $B = \tau_j \sum_i R_{ij}^{-k} \bar{l}_{ik} + w_S \big[\bar{\beta}_k \sum_i W_{ii} y_{ij} z_i^{-k}$ $- \overline{\beta_k^2} \sum_i W_{ii} y_{ij} (\bar{z}_{ik} - y_{ij} \bar{f}_{jk}) \big]$ (sequential over genes)
$q(\beta_k)$	$A = w_S \sum_i W_{ii} \overline{l_{ik}^2}$ $B = w_S \sum_i W_{ii} z_i^{-k} \bar{l}_{ik}$	$A = w_S \sum_i W_{ii} (\bar{z}_{ik}^2 + V_{ik}^Z)$ $B = w_S \sum_i W_{ii} z_i^{-k} \bar{z}_{ik}$
τ_j	$n / \sum_i \bar{R}_{ij}^2$	$n / \sum_i \bar{R}_{ij}^2$
Order	l_k, f_k, β_k per k; then τ	β_k, l_k, f_k per k; then τ; rescale

What changed from the March LB derivation. Moving from LB to YFB (i) removes the survival term from the L update, (ii) adds one to the F update, which makes that update a sequential sweep over genes because the score $\bar{z}_{\cdot k}$ couples the genes, (iii) replaces $\bar{l}_{ik}$ by the score $\bar{z}_{ik}$ in the β update, with V_{ik}^Z playing the role of $\mathrm{Var}_q(l_{ik})$, (iv) adds score standardization, and (v) replaces the latent scores by a projection at prediction. The Taylor expansion, the EBNM reduction and the τ update are unchanged. The March derivation is correct for LB. The YFB derivation document lists the corrections made to the April YFB pieces.

4 Where the Balance Between Genomics and Survival Comes From

Survival influences the factorization only through the dual-source update, and there only through its share of the precision, $\rho = w_S A^{\mathrm{surv}} / A^{\mathrm{gen}}$. The pseudo-observation is the precision-weighted average of the genomics-only and survival-only estimates, with weight $\rho/(1+\rho)$ on survival.

LB, patient score l_{ik}.

$$\rho_{ik}^{\mathrm{LB}} = \frac{w_S W_{ii} \overline{\beta_k^2}}{\sum_j \tau_j \overline{f_{jk}^2}}.$$

Fix the scale so that $\sum_i \bar{l}_{ik}^2 \approx n$. With $d_k^2 = \|\bar{l}_{\cdot k}\|^2 \|\bar{f}_{\cdot k}\|^2 = n p v_k$ we get $\|\bar{f}_{\cdot k}\|^2 \approx p v_k$. With W_{ii} averaging D/n,

$$\rho^{\mathrm{LB}} \approx \frac{w_S \overline{\beta_k^2}\,(D/n)}{\bar{\tau}\, p\, v_k}.$$

YFB, gene loading f_{jk}. From the YFB derivation (Section on balance),

$$\rho_{jk}^{\mathrm{YFB}} = \frac{w_S \overline{\beta_k^2} \sum_i W_{ii} y_{ij}^2}{\tau_j \sum_i \overline{l_{ik}^2}} \approx \frac{w_S \overline{\beta_k^2}\,(D/n)}{\tau_j\, p^2 v_k^2}.$$

Both ratios are unchanged by the scale symmetry, and β_k is per SD of the score in each. They depend on n only through the event fraction D/n.

Reading the two ratios

- Both fall with p. The YFB weight falls faster, as $1/p^2$ against $1/p$, because in YFB a unit-SD score $Y\boldsymbol{f}_k$ forces small loadings and large patient scores, and the genomics precision of f_{jk} grows with $\|\boldsymbol{l}_k\|^2$. The loadings themselves are also small, of order $1/(p\sqrt{v_k})$, while the survival-only estimate does not shrink with p, so the relative change survival makes in a YFB loading falls roughly as $1/p$, the same order as in LB.
- Both rise as the program's variance share v_k falls, more steeply in YFB. Supervision acts mostly on weak programs in both models.
- The LB form is the "survival cannot compete" imbalance found in spring 2026 ($\rho \approx 1/(p\bar{\tau})$ when the other factors are of order one). The April YFB note claimed the ratio becomes $O(1)$ in YFB. That claim was wrong.

Measured shares (context).

Model	Setting	Survival share	Source
YFB	sim., $p = 50$, planted	0.79–1.00	S-9 findings
YFB	sim., $p = 2{,}000$, planted	0.01–0.20	S-9 findings
YFB	PDAC training, $p \approx 2{,}200$	10^{-8} to 10^{-6}	DECISIONS.md 2026-11-04

In the S-9 simulation the share was 0.20, 0.05 and 0.01 for planted variances of 0.25%, 1% and 5% at $p = 2{,}000$, the ordering the ratios predict. The S-9 trace records $\rho/(1+\rho)$. The LB simulation of 2026-11-06 showed no gain from supervision over a two-step Cox fit on the fitted scores, consistent with a small share.

Does weighting help? A weight changes the fitted factors only if it moves ρ to order one for some program. In YFB that happens first for low-variance programs, and the near-threshold simulation (S-14) is where supervision outperformed two-step fitting. For dominant programs the weight needed exceeds the range where the fit stays stable.

5 What Happens When Survival Is Given More Weight

The two models fail differently under a large weight.

- **YFB.** The survival-only estimate of $\boldsymbol{f}_k$ is a weighted least squares fit of $\boldsymbol{z}^{-k}$ on Y with $p > n$ unknowns, so it has exact solutions. As ρ grows, the loadings can follow a gene combination that orders the training times perfectly. S-9 lost convergence and the program at $w_S \geq 40$ with $p = 2{,}000$, but its coefficients stayed near their true values, so separation is a hypothesis and the cause has not been identified.
- **LB.** Each patient's score l_{ik} has its own survival pseudo-observation. Earlier attempts to strengthen it failed: multiplying the survival term of the L update by p diverged, and dividing the genomics term by p collapsed L and F to zero (DECISIONS.md, 2026-07-12). At a weight that does converge, each training score moves toward its own outcome. The training scores then carry outcome information that a new patient's scores cannot have, because the new patient has no outcome (Section 6), and training performance overstates test performance.

Options for both models (details and status in the YFB derivation, Section "Strengthening Supervision"):

1. Choose the weight from data by cross-validated partial log-likelihood. In YFB this picked w_S between 1 and 10 in simulation and never a non-converging weight (S-13).

2. Keep the prior on $\boldsymbol{\beta}$ at full weight while the likelihood weight rises (the survival-block objective). This limits the coefficient blow-up that comes with separation.

3. Normalize the two losses and mix them with α, as DeSurv does, with α chosen by cross-validated C-index. In the Bayesian objective this is the same as a data-chosen w_S.

4. Divide the genomics term by p. Multiplied through by p, this is $w_S = p$ with every KL term also multiplied by p. In LB it collapsed L and F to zero. In YFB it leaves a $1/p$ dependence, and $w_S \approx 2{,}000$ is far beyond the weights that converged.

5. Reduce p (a smaller gene set) to raise every program's share. Any outcome-based screen has to be repeated inside cross-validation.

DeSurv for comparison

DeSurv minimizes $(1-\alpha)L_{\text{NMF}}(W,H) - \alpha L_{\text{Cox}}(W,\boldsymbol{\beta})$ with scores $Z = W^\top X$. The survival gradient acts on the gene programs W, and H enters only the reconstruction. Structurally this is the YFB parameterization: W plays the role of F and H that of L. DeSurv selects (k, α, λ) by Bayesian optimization of the cross-validated C-index. It selected $\alpha = 0.334$ in PDAC, and its survival signal concentrated in one program with 7% of the variance. The differences from YFB are the penalized point estimates in place of empirical-Bayes posteriors, and a tuned survival weight in place of $w_S = 1$. The comparison suggests that a data-chosen survival weight, not a different parameterization, is the main step still missing in YFB.

6 Prediction for a New Patient

YFB. $\hat{\eta}^* = \sum_k (\boldsymbol{y}^{*\top} \bar{\boldsymbol{f}}_k)\bar{\beta}_k$. Training and test use the same formula.

LB, current code. `predict_ssbmf_lb()` computes the least-squares projection $\boldsymbol{l}^* = (\bar{F}^\top \bar{F})^+ \bar{F}^\top \boldsymbol{y}^*$ and $\hat{\eta}^* = \boldsymbol{l}^{*\top}\bar{\boldsymbol{\beta}}$. It uses posterior means only and ignores τ, the loading variances and the prior g_l, so it can return negative scores under a nonnegative prior.

LB, fold-in (Yusha, 2026-10-30). Freeze $q(F)$, τ, $q(\boldsymbol{\beta})$ and the fitted priors $\hat{g}_{l_k}$. For the new patient, maximize the objective over $q(\boldsymbol{l}^*)$ alone. The patient has no outcome, so only the genomics term enters:

$$A_k^* = \sum_j \tau_j \overline{f_{jk}^2}, \qquad B_k^* = \sum_j \tau_j \Big(y_j^* - \sum_{k' \neq k} \bar{l}_{k'}^* \bar{f}_{jk'} \Big) \bar{f}_{jk}, \qquad q(l_k^*) \propto \mathcal{N}\big(B_k^*/A_k^*;\ l_k^*,\ 1/A_k^*\big)\, \hat{g}_{l_k}(l_k^*),$$

cycling over k until the moments stop changing. Each step is a closed-form normal-means posterior under a fixed prior, as in the second stage of EBNM. No refit is needed.

Experiment (2026-11-06). The fit object does not store $\hat{g}_{l_k}$, so the priors were recovered by evaluating one more L step at the final state. Fold-in then used `ebnm(..., g_init = g, fix_g = TRUE)` and converged in 6–10 sweeps. Over eight train/test splits:

- Test scores against the true scores: fold-in was closer in 8 of 8 splits (mean $|r|$ 0.735–0.746 against 0.677–0.723 at the stronger signal).
- Test C-index: fold-in and projection were tied within seed noise (differences from -0.017 to $+0.035$).
- Folding in the training patients reproduced their fitted scores to within 0.7–2.1% (column correlations 1.00). The difference is the survival contribution the training scores received, which was small here.

The trade-off fold-in exposes

Fold-in is the right way to score a new patient under LB, and it avoids double use of the outcome. Its gap from the training scores, however, equals the survival contribution to those scores. When supervision is weak the gap is negligible, and so is supervision. When supervision is strong the training scores move toward outcomes that fold-in cannot use, and training fit overstates test performance. YFB has no such gap, since its risk score is a function of $\boldsymbol{y}$ alone.

Building fold-in into LB. Store $\hat{g}_{l_k}$ (and the weight used) in the fit object at the last L update, and add a fold-in function beside `predict_ssbmf_lb()`. This is a small change. Fold-in inherits any program that collapsed during fitting.

7 LB Implementation Review

`code/fit_ssbmf_lb.R` and its update functions were compared with the March derivation. The update formulas, the update order and the once-per-iteration Taylor expansion agree. The full test suite passed (761 of 761), as did the LB subset (228 of 228). In simulation all 40 fits converged in 6–88 iterations.

Sign check inverted correctly oriented fits (fixed 2026-11-06)

After convergence, `fit_ssbmf_lb()` flips $\bar{\boldsymbol{\beta}}$ when the training concordance is below 0.5 (`sign_correction = TRUE`, the default). It calls `concordance(Surv(time, status) ~ eta_train)` without `reverse = TRUE`. `survival::concordance` then treats larger η as longer survival, so a correctly oriented Cox risk score reads below 0.5 and is flipped. In the simulation, 8 of 8 default fits ended with inverted risk (training C 0.21–0.38, against 0.62–0.79 without the flip). The YFB fitter was corrected for this on 2026-09-04, but LB was not. The flip also sets `EBeta2 <- EBeta^2`, which discards the posterior variance, although a sign change leaves the second moment unchanged. A global flip of $\boldsymbol{\beta}$ alone is not a symmetry of the model in any case. The check now uses `reverse = TRUE` and keeps `EBeta2` (`tests/test_lb_sign_correction.R`). The LB benchmark runner and the multistart diagnostic had computed C without `reverse = TRUE`, which cancelled the inversion; both were corrected with it, so their reported C-indices are unchanged. Saved LB risk scores and `EBeta` from earlier runs with the default have the opposite sign.

Item	Derivation	Code
Survival weight	none (weight 1)	genomics scaled by $1-\alpha$ and survival by α (default 0.5). At $\alpha = 0.5$ this tempers both likelihoods equally, which changes s but not x in the L update. Other values act as a survival weight $\alpha/(1-\alpha)$ plus tempering. No α reproduces the derivation
Cox weight W_{ii}	exact diagonal	$\hat{\theta}_i \hat{H}(t_i)$, which drops a small term and is never below the exact value (same as YFB)
Convergence	change in $\bar{L}$, $\bar{\beta}$	relative ELBO change; the monitored ELBO multiplies the survival term by p (`norm_convention = "per_p"`), which the updates do not
Orthogonalization	a note describes an SVD rotation every 10 iterations	not in the current code; the note is out of date
Collapsed programs	–	all 40 fits had at least one program with $\bar{L}$ and $\bar{F}$ exactly zero (two in 3 fits), likely from the `pmax(., 0)` SVD start under the nonnegative prior. There is no pruning inside the fitter

8 Strengths, Weaknesses and Recommendation

	LB	YFB
Strengths	Outcome kept out of the expression model; each score has its own survival information; the March derivation is checked	Exact risk scores by projection; no train/test gap; same structure as DeSurv; supervision acts on the gene programs that are reported
Weaknesses	New patients need fold-in; train/test gap grows with supervision strength; collapsed programs	Y used as outcome and covariate; survival weight falls as $1/p^2$; no convergence at large weights
Current status	runs; supervision weak; no gain over two-step in simulation	supervision effective near the detection threshold (S-14); weight choice by CV is the open step

Recommendation

1. Keep YFB as the primary model. Its exact projection scoring matches the goal of external prediction, and its open problem, how much weight to give survival, has a data-driven answer under test (CV-chosen w_S).

2. Keep LB as an alternative parameterization. With the sign check fixed, store $\hat{g}_{l_k}$ and add fold-in prediction. Then both models can be run on the same simulations and compared under matched weights.

3. Report supervision strength with the measured precision share ρ for every fit, so that a supervised result can be told apart from a two-step one.